Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Proje soruları: SRS belgesinin formatı, proje bilgilerinin ders sistemine konması, sistem testi aracı araştırması.
- Test süreci adımlarının kısa tekrarı.
- Yazılım ölçümü tekrarı: ölçme ihtiyacı, Halstead, McCabe, güvenirlik, büyüklük ölçütleri (LOC, function point), CK metrikleri.
- Dersin o güne kadarki akışının özeti ve test tiplerinin test adımlarıyla ilişkisi.
- Test yönetimi: test spesifikasyonu, test planı, test senaryosu, test çalışma ekibi, optimum test miktarı.
- Yazılım yaşam döngüsü boyunca test etkinlikleri.
- Teslim sonrası kalite: yazılım bakımı (maintenance), bakıma elverişlilik, yazılım konfigürasyon yönetimi, sürüm yönetimi (release management).
- Vize zamanı ve not dağılımı.

Hocanın Özellikle Vurguladığı Kısımlar

- IEEE 830 formatı yol göstericidir. Başlıklara sıkı sıkıya bağlı kalmak gerekmez; özellikle 2. ve 3. bölümler özetlenebilir.
- Fonksiyonel gereksinimler madde madde de yazılabilir, use case belgeleriyle de gösterilebilir. İkisi de geçerlidir.
- Kullanıcı testinde yalnızca kara kutu testi yapılır. Birim testinde ağırlıklı olarak saydam kutu testi yapılır.
- Bir yazılımda ne yalnızca saydam kutu ne de yalnızca kara kutu testi yapılır; ikisi birlikte yürütülür.
- “Saydam kutu” adı “beyaz kutu”dan daha doğrudur, çünkü iç yapı görülür (glass box).
- Test ekibi, yazılımı tasarlayan ve geliştiren grubun içinde olmamalıdır. Teste bağımsızlık önemlidir.
- İyi bir testçi mükemmelliğe ulaşılamayacağını ve ne zaman yeterli olduğunu bilir. Her projede optimum test miktarı farklıdır.
- Projede mutlaka olması gerekenler: test planı, testlerin tek tek nasıl yapıldığını açıklayan rapor ve beklenen sonuçlarla gerçek sonuçların karşılaştırılması.
- LOC ölçmesi kolay olduğu için dezavantajları bilinerek yine de kullanılır.

Kısa Tekrar Notları

- Test süreci adımları: birim testi, bütünlük testi, sistem testi, kullanıcı testi.
- Beta testi kullanıcı testinin bir aşamasıdır ve kara kutu testidir.
- McCabe: akış grafındaki kapalı bölgeler ile dış bölgenin toplamı modülün karmaşıklığını verir. Modülde bu değerin en çok 10 olması önerilir.
- Güvenirlik: geçmiş bilgilere ve teste dayanan, toplam işletim süresinin ne kadarında başarılı çalışıldığını kestiren ölçü.
- LOC dezavantajları: dile, yazım biçimine, algoritmaya ve yeniden kullanıma göre değişir; iş gücüyle her zaman doğru orantılı değildir.
- Test yönetimi adımları: planlama, tasarım, uygulama (testlerin çalıştırılması), raporlama. Süreç geri beslemelidir.

- Test spesifikasyonu = test planı + test senaryoları + test işlemleri + beklenen ve gerçek sonuçlar.
- Bakım işlerinin dağılımı (hocanın verdiği): %21 hata düzeltme, %25 iyileştirme, %50 uyarlama, %4 diğer.
- Yazılım konfigürasyonu: programlar, belgeler, veri yapıları ve veritabanı elemanlarının bütünü.

Detaylı Açıklamalar

Proje Soruları

- SRS teslimi bir hafta ertelenmişti. Belge önümüzdeki hafta (17 Kasım 2020) her grup adına bir temsilci tarafından gönderilecektir.
- Bir öğrenci IEEE formatının başlıklarına uymanın zorunlu olup olmadığını sordu. Cevap: hayır. Format yol göstericidir. Derste gösterilen öğrenci projesi örneği de özetlenmiş bir formattır.
- Bir öğrenci proje adımlarının ders sistemine yazılmasını istedi. Hoca, gereksinim analizinden sonraki adımları maddeler halinde kısaca sisteme yazacağını söyledi. Bazı slaytları da özetleyip ekleyecektir.
- Projede kullanılan dil için birim testi olanaklarını ve bir sistem testi aracını (performans, güvenlik veya dayanıklılık testi yapan) şimdiden araştırın. Bunlar SRS ile birlikte gönderilmeyecek; test aşamasında kullanılacaktır.
- Yazılımın kimin geliştirdiği önemli değildir (kendi yazılımınız, başka bir yerden bulunan ya da bir arkadaşınızın projesi). Önemli olan grubu sabitlemek ve seçilen yazılımın gereksinim analiziyle devam etmektir.
- Grup bulamayan biri tek başına çalışacağı konuyu hocaya yazmalıdır.
- Test belgeleri (test planı, test senaryoları, test verisi, test sonuç raporu) dönem sonundan 3 hafta önce teslim edilecektir.

Yazılım Ölçümü Tekrarı

- Geliştirme işleri ve ürün ölçülebilirse yazılım geliştirme süreci daha iyi yönetilir ve kontrol edilir.
- Yöneticiler ve yazılım mühendisleri aynı şeyleri farklı açılardan ister.
  - Yöneticiler: üretim maliyeti, hangi aşamada ne kadar iş gücü harcandığı ve personel verimliliği, ücretlerin belirlenmesi, ürün ve araçların kıyaslanması.
  - Yazılım mühendisleri: süreç ölçümleriyle kaliteyi izlemek, tasarım değişikliklerini ve test hatalarını görmek, performans ve kalite gereksinimlerini belirlemek, gelecekteki ürünlerde tasarım yolunu seçmek, arşiv verisi oluşturmak.
  - Sertifika için de ölçme yapılır: hem ürüne hem yazılım evine sertifika almak için ürün ve süreç özellikleri ölçülür. Örnek ölçüt: modüllerin 100 kod satırını geçmemesi.
- Algoritma karmaşıklığı da bir ölçüttür; algoritma derslerinde işlendiği için tekrar edilmedi.
- Halstead metriği: program çok satırlı ama düz akışlı (okuma, yazdırma) olabilir ya da az satırda çok hesaplama içerebilir. İşleç ve işlenen sayıları ile döngü içi yinelemelerden program uzunluğu, hacmi ve gücü hesaplanır.
- McCabe metriği:
  - Tasarım aşamasında kullanılır ve kontrol akışı hakkında bilgi verir.
  - Döngüler ve if blokları gibi kontrol deyimleri akış grafında düğüm ve kenar olarak gösterilir.
  - Pressman'ın ortalama alma örneğinde R1 dış bölge, R2-R6 kapalı bölgelerdir. Bölge sayısı döngüsel karmaşıklığı ve bağımsız yol sayısını verir.
  - Bağımsız yol sayısı 10'dan fazla olursa modül çok karmaşıktır ve test etmek zorlaşır. Modülü alt parçalara bölmek iyi olur.
- Güvenirlik:
  - Bir yazılımda mutlaka hata kalır. Güvenirlik bu hatanın miktarıyla ilgilidir.
  - Modeller önceki kullanımların istatistiklerine dayanır: toplam hata sayısı, güvenirlik düzeyi, revizyondan sonra kalan hata sayısı.
  - 1980'lerden beri 100'e yakın model geliştirilmiştir (Jelinski, büyük olasılıkla (slayttan) Musa Basic vb.).

  - Model seçimi: ölçmesi kolay mı, yazılımın tümü gerekli mi, kolay anlaşılır mı, dış etkenlerden etkileniyor mu?
  - Güvenirliği sağlamak için: gereksinimlerde işlevsel yapıyı belirlemek, hataları önem derecesine göre (ölümcül, büyük, orta vb.) tanımlayıp sınıflandırmak, müşterinin güvenirlik beklentisini tanımlamak; tasarım ve kodlamada bileşenler arası güvenirliği belirlemek ve hatalı kodları incelemek; sistem testinde güvenirlik testleri ve ek testler; teslimde güvenirliğin yeniden kontrolü ve müşteri memnuniyetinin izlenmesi.

Büyüklük Ölçütleri

- Dolaylı ölçüm: işlev noktası (function point). Dosya (veritabanı tablosu), girdi, çıktı ve ara birim sayıları sayılıp hesaplama yapılır. Karmaşıklık değil, büyüklük ölçüsüdür.
- Doğrudan ölçüm: kod satırı (LOC) veya bin kod satırı (KLOC).
- LOC'un dezavantajları:
  - Yazım biçimi satır sayısını değiştirir. Bir öğrencinin örneği: parantezleri ayrı satırlara yazmak satır sayısını artırır.
  - Programlama diline göre değişir. C'de çok satır tutan iş başka bir dilde az, Assembler'da daha çok satır tutar. Bu farkı gidermek için diller arası satır karşılık tabloları hazırlanmıştır.
  - Daha iyi bir algoritma tasarlanırsa aynı iş daha az satırla yazılır. Yeniden kullanım (reusability) da satır sayısını düşürür.
  - Bu nedenle programcının harcadığı emek satır sayısıyla her zaman doğru orantılı değildir.

CK Metrikleri Tekrarı

- Sınıfın ağırlıklı metot sayısı (WMC): sınıftaki metotların karmaşıklıkları toplamıdır. Karmaşıklık döngüsel karmaşıklık da algoritma karmaşıklığı da olabilir. Hem kalite hem harcanan emek için önemlidir.
- Kalıtım ağacı derinliği (DIT): sınıfın köke uzaklığıdır. Türetilmemiş sınıf için 0'dır. Çoklu kalıtımda en uzak köke olan uzaklık alınır.
  - Derindeki sınıflar daha çok metot devralır; davranışlarını tahmin etmek zorlaşır ve tasarım karmaşıklığı artar.
- Alt sınıf sayısı (NOC): çok fazla alt sınıf kalıtımın yanlış kullanıldığını gösterebilir. Bu sınıfların metotları daha çok test gerektirir; test bütçesi artar.
- Nesne sınıfları arası bağımlılık (CBO): bir sınıfın nitelik veya metotları kalıtım olmadan başka bir sınıfta kullanılıyorsa bağımlılık vardır.
  - Aşırı bağımlılık modüler tasarıma zarar verir, değişime duyarlılığı ve bakım zorluğunu artırır, test maliyetini yükseltir.
  - Dengenin ne olacağı tasarım planlamasında belirlenir.
- Sınıfın tetiklediği metot sayısı (RFC): bir mesaj çok sayıda metodu tetikliyorsa sınıfın testi ve hata ayıklaması zorlaşır.
- Metotlardaki uyum eksikliği (LCOM): sınıfın metotları ve yerel nitelikleriyle kendi içinde ne kadar uyumlu olduğunu gösterir. Birden fazla tanımı vardır.
- CK dışında MOOD metrikleri gibi başka nesneye yönelik metrik kümeleri de vardır.

Dersin Akışı ve Test Tipleri

- Bugüne kadar işlenenler:
  - Yazılım mühendisliğinde kalitenin yeri.
  - Kalite özellikleri ve McCall kalite modeli.
  - Kaliteyi sağlama aktiviteleri: standartları belirlemek, süreç modelini seçmek, formal ve kurallı geliştirmek, gözden geçirme (review) ve inceleme (inspection) yapmak.
  - Test süreci adımları, V model, doğrulama ve geçerleme, her adımda test verisi ve senaryo hazırlama.
  - Test tipleri: kara kutu, saydam kutu ve gri kutu.
  - Değerlendirmenin dayanağı olan yazılım metrikleri.
- Derste sorulanlar:
  - Saydam kutu testi ağırlıklı olarak hangi adımda kullanılır? Birim testinde.

  - Beta testi hangi seviyede ve hangi tipte yapılır? Kullanıcı testi seviyesinde, kara kutu testi olarak; kullanıcı iç yapıya değil dışarıdan bakar.

Test Yönetimi

- Projede gereksinim spesifikasyonundan sonra test yönetimi yapılacaktır: planlama, tasarım, testlerin uygulanması ve test raporunun hazırlanması.
- Bu bölümün hedefleri: test spesifikasyonu belgesinin içeriğini incelemek, test planının içeriğini öğrenmek, test senaryosu hazırlamak.
- Test yönetimi geri beslemeli bir süreçtir.
- Test hata mesajları büyük olasılıkla (slayttan) doküman testi altında sınıflanır. Hata mesajının içeriği belge olarak test edilir; doğru mesajın ekranda görünmesi ise kod testi gerektirir.
- Test spesifikasyonunu IEEE'nin yazılım test belgelendirme standardı (Software Test Documentation) tanımlar.
- Test spesifikasyonu geliştirme döngüsü: yaz, incele, gerekirse düzelt, tekrar çalış. Her sürüm bir temel çizgi (baseline) olarak izlenir; bu kavram konfigürasyon yönetiminde tekrar ele alınacaktır.

Test Spesifikasyonu (Projede İstenen Rapor)

- Test planı: hangi tip testler yapılacak, zamanlama, gider, test ortamı.
- Test senaryoları: hangi test için hangi test verisi kullanılacak.
- Test işlemleri: her testin tanımı, test edilen modüller, kullanılan özel araç ve teknikler.
  - Örnek: Java ile yazılmış bir modülün birim testi JUnit ile yapılır; raporda hangi modülün JUnit ile test edildiği yazılır.
  - Gider kısmı projede boş kalabilir.
  - Test senaryoları ayrı bölüm yerine test işlemlerinin içinde verilebilir: her test hangi senaryoyla çalıştırıldı?
- Beklenen sonuç (gereksinim analizinden çıkarılır) ve gerçek test sonucu.
- İsterseniz özet bir sonuç raporu da eklenebilir.

Test Planının İçeriği

- Slayttaki maddeler:
  - Test stratejisi ve test edilecek öğeler: hangi test adımlarından geçilecek, gereksinim raporuna atıfla hangi elemanlar test edilecek?
  - Doğrulama yöntemleri: doğrulama ve geçerleme için ne kullanıldı?
  - Testlerin tamamlanma kriterleri: örneğin hata oranı hangi değere inince test durdurulacak?
  - Hata ve test sonuç raporlama adımları.
  - Test sorumlulukları: ekip yapısı ve görevler.
  - Test ortamı: kullanılacak test yazılımı araçları.
  - Eleman ve eğitim ihtiyacı: kaç testçi gerekli, eğitim gerekli mi?
  - Test takvimi: örneğin Gantt diyagramı.
  - Risk yönetimi: test sürecinde bir sorun çıkarsa ne yapılacak?
  - Test çıktıları: beklenen çıktılar.
- Bu ayrıntılar değişebilir. Proje planında daha az madde kullanılabilir.

Test Senaryosu

- Bir test senaryosunda bulunanlar: adı ve kimliği, test ettiği gereksinimler, amacı, ön koşullar, varsayımlar, test girdileri, adımlar, beklenen sonuçlar.
- Birim veya bütünlük testinde (saydam kutu) senaryo iç yapıyı ya da tasarıma uygun bütünlemeyi kontrol etmek için hazırlanır.
- Kullanıcı testinde (geçerleme) senaryo dışarıdaki kullanıcının bakış açısıyla, kullanıcıyla birlikte hazırlanır.
  - Örnek: öğrenci bilgi sisteminde öğrenci işleri çalışanlarıyla birlikte, çift anadal yapan öğrencinin ders seçimi veya sınav değerlendirmesi gibi farklı girdiler için senaryolar oluşturulur.

Test Çalışma Ekibi ve Optimum Test Miktarı

- Test ekibinin görevleri: planlanan testlerin hazırlığını yapmak, testleri yürütmek, bulunan hataları yazılımcıya bildirip izlemek, sonuçları raporlamak.
- Şekil 7.3 (her projede optimum test miktarı farklıdır):
  - Yatay eksen test miktarıdır.
  - Test maliyeti eğrisi test miktarı arttıkça önce yavaş, sonra hızla yükselir.
  - Kaçırılan hata (bug) sayısı eğrisi test miktarı arttıkça düşer.
  - İki eğrinin kesiştiği nokta optimum test miktarıdır. Bu noktanın ötesi "test dışı" bırakılır.
- Amaç: kaçırılan hata sayısını en aza indirirken test maliyetini çok yükseltmemek.

Yazılım Yaşam Döngüsü Boyunca Test Etkinlikleri

Aşama	Test etkinliği
Planlama	Sistem test planı, yazılım kalite planı
Gereksinim analizi	Alt sistem sınama (test) planları
Tasarım	Modül test planı, büyük olasılıkla (slayttan) test tanımları ve test eğitim el kitapları
Gerçekleme	Birim testi, bütünlük testi, sistem testi
Kurulum	Kullanıcı testi, test raporları

- Tasarımda McCabe ölçüsüyle algoritma yolları belli olur; her modülün nasıl test edileceği o sırada planlanabilir.
- Sistem testi modül bazında da yapılabildiği için gerçekleme sırasında başlayabilir.
- Kurulumda kullanıcıya teslim edilir; kullanıcı testi yapılır ve test raporları tamamlanır.
- Bu adımlar seçilen süreç modeline göre değişir. Bazı modellerde test planları en başta, bazılarında sona doğru hazırlanır.

Teslim Sonrası Kalite: Yazılım Bakımı

- Kaliteyi sağlama aktivitelerine teslimden sonra bakım ve konfigürasyon yönetimi eklenir.
- Yazılım bakımı (software maintenance): sonradan görülen hataların düzeltilmesi, yazılımın iyileştirilmesi, uyarlanması ve geliştirilmesi.
- Hocanın verdiği bakım işi dağılımı:
  - %21 hata düzeltme.
  - %25 iyileştirme: yeni araçlarla kaliteyi artırma, yenilikler.
  - %50 uyarlama: gerçek hayattaki ortam değişikliklerinin yazılıma yansıtılması.
  - %4 diğer.
- Büyük bir yazılımda testte bulunamamış hatalar (işlem, performans, tasarım hatası) sonradan çıkabilir.
- Yeteneği artırmak veya kullanımı kolaylaştırmak için birkaç modülde değişiklik gerekebilir.
- Günün koşullarına yetmeyen yazılım gereksinim analizi, tasarım ve test adımlarıyla yeniden geliştirilebilir.
- Teslim sonrası hata hem kullanıcı için (bakım sözleşmesi maliyeti) hem geliştirici için (güven kaybı) pahalıdır.

Bakım Maliyetini Azaltmak

- Geliştirme sırasında bakıma elverişlilik (maintainability) sağlanmalıdır:
  - Deneyimli ve dikkatli yazılım mühendisleri.
  - Anlaşılabilir sistem yapısı.
  - Standart programlama dilleri ve işletim sistemleri.
  - Standartlara uygun belgeler.
  - Test programlarından ve yazılım/test araçlarından yararlanmak.
  - Hataları tasarım aşamasında bulmak için tüm gayreti göstermek.

- Bakıma elverişliliği belirleyen kalite faktörleri: test edilebilirlik, basitlik, değiştirilebilirlik (altında modülerlik vardır), taşınabilirlik (başka işletim sistemine aktarılabilme), güvenirlik, esneklik.

Yazılım Konfigürasyon Yönetimi

- Yazılım mühendisliği ürünleri (programlar, belgeler, veri yapıları, veritabanı elemanları) topluca yazılım konfigürasyonudur. Türkçede yazılımın düzeni veya yapısı da denir.
- Yazılım konfigürasyon maddesi (Software Configuration Item, SCI): konfigürasyon yönetiminin uygulandığı tek tek yazılım elemanıdır.
- Konfigürasyon yönetimi işlemleri:
  - Konfigürasyon tanımlama (configuration identification): her modülün, veritabanı elemanının, belgenin ve test senaryosunun tanımlanması.
  - Konfigürasyon değişiklik kontrolü.
  - Konfigürasyon raporlama.
- Konfigürasyon yönetim planının içeriği standartlarla tanımlanmıştır. Hoca ISO 9004'te de ilgili tanım bulunduğunu söyledi.
- Bakımdaki her değişiklikte konfigürasyon yönetimi sağlanmalıdır.
- Sürüm yönetimi (release management) konfigürasyon yönetimiyle birlikte ele alınır. Sektörde önemi artmıştır; sonraki derslerde işlenecektir.

Vize ve Not Dağılımı

- Vize için ortak bir sınav haftası henüz belirlenmemişti. Hoca normalde 8. haftada yaptığını, bu yıl projeyle çakışmaması için 9. haftayı düşündüğünü söyledi. Kesin bilgiyi bölümden öğrenip sonraki hafta bildirecektir.
- Sınavın ders sistemindeki sınav modülüyle yapılması düşünülüyordu.
- Geçici not dağılımı: %30 vize, %30 proje, %40 final. %40 final zorunluluğu yoksa projenin ağırlığı artırılabilir.

Ders sonrası notlar

- Slaytta "(830-1993) IEEE Standard for Software Test Doc." yazıyordu. Hoca da numaranın yanlış olduğunu söyledi. IEEE 830 gereksinim spesifikasyonu (SRS) standardıdır. Yazılım test belgelendirme standardı IEEE 829'dur (Standard for Software Test Documentation).
- Bir öğrenci SRS formatını "IEEE 802" diye andı. IEEE 802 ağ standartlarıdır; SRS formatı IEEE 830'dur.
- Bakım dağılımı için yaygın kaynak olan Lientz ve Swanson çalışmasında oranlar farklıdır: iyileştirme (perfective) yaklaşık %50, uyarlama (adaptive) yaklaşık %25, hata düzeltme (corrective) yaklaşık %21, önleyici/diğer (preventive) yaklaşık %4. Hocanın derste söylediği oranlarda iyileştirme ile uyarlamanın yerleri değişmiş olabilir.
- Konfigürasyon yönetimi için daha doğrudan kaynaklar: yazılım konfigürasyon yönetim planı için IEEE 828, genel konfigürasyon yönetimi rehberi için ISO 10007. ISO 9004 genel kalite yönetimi rehberidir.
- Konfigürasyon yönetiminin dört klasik işlemi tanımlama, değişiklik kontrolü, durum raporlama (status accounting) ve denetimdir (audit). Altyazıda değişiklik kontrolü ile raporlama arasında boşluk vardır; denetim adımı bu boşlukta anlatılmış olabilir.
- Whisper altyazıda "makyayip/MacGabe" McCabe, "hashtag metriği" Halstead, "Çitember Kemerer" Chidamber-Kemerer, "REEE" IEEE olarak düzeltilmelidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.